Computer Organization and Assembly Language

Week 03 — Listing Files, the iAPX 88 Register Architecture and the Segmented Memory Model

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 03

Outline

1. Quick Review
2. The NASM Listing File and Opcodes
3. The Intel iAPX 88 Architecture
4. Register Architecture of the iAPX 88
5. The Segmented Memory Model
6. Physical Address Calculation

Quick Review: What Was Done Last Week?

- **Organization vs Architecture** — implementation vs what the programmer sees.
- **Internal organization** of a computer: CPU, memory, I/O and the three buses.
  - **Address bus** — unidirectional, selects the cell.
  - **Data bus** — bidirectional, carries the data.
  - **Control bus** — bidirectional, carries the intent (read / write) and status.
- **Memory**: number of cells $\times$ cell width; RAM vs ROM; why cache and registers exist.
- **Registers**: accumulator, index / pointer, flags and the instruction pointer.
- **Fetch–Decode–Execute** cycle; opcodes, mnemonics and the assembler.
- **First program** assembled with NASM, executed and traced in DOSBox with AFD; **little endian** storage.

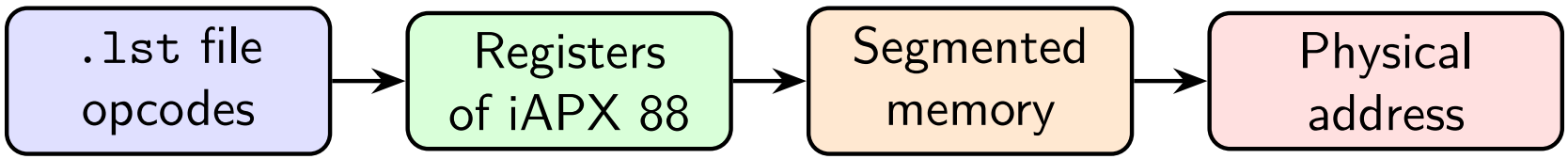
Where We Are Heading Today

From last week

We saw *that* the assembler produces opcodes and addresses.

This week

We see *how* those addresses are formed, which registers exist inside the 8088, and how a 16-bit CPU reaches 1 MB of memory.



NASM Listing File (`.lst`)

```
 1 [org 0x0100]
 2
 3 ; start of code
 4
 5 00000000 B80500 mov ax, 5 ; move the constant 5 into register ax
 6 00000003 BB0A00 mov bx, 10
 7
 8 00000006 01D8 add ax, bx ; add value of bx into the value of ax
 9
10 00000008 BB0F00 mov bx, 15 ; move constant 15 into register bx
11 0000000B 01D8 add ax, bx
12
13 0000000D B8004C mov ax, 0x4c00 ; exit ..
14 00000010 CD21 int 0x21 ; .. is what the OS should do for me
```

line no. | address (offset) | opcode bytes | source line

How Are the Addresses Calculated?

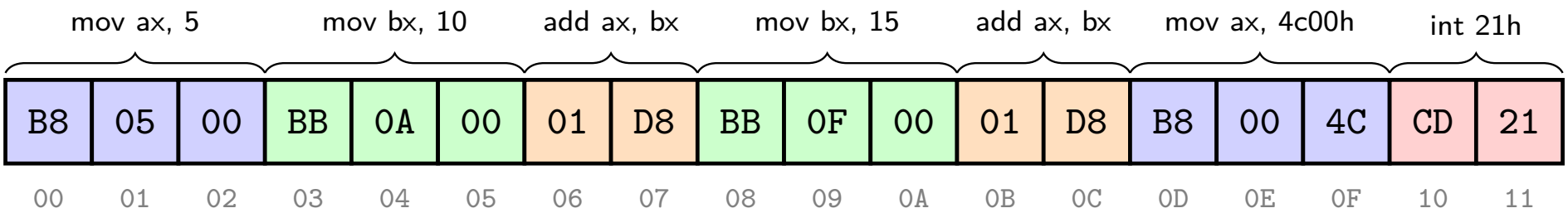
- The address column is simply a **running byte count** of the generated code, starting at 00000000.
- Each instruction occupies as many bytes as its encoding needs; the next address = current address + instruction length.

$$\text{Address}_{n+1} = \text{Address}_n + \text{Length}(\text{Instruction}_n)$$

Addr	Opcode	Len	Instruction	Next
00000000	B8 05 00	3	mov ax, 5	00000003
00000003	BB 0A 00	3	mov bx, 10	00000006
00000006	01 D8	2	add ax, bx	00000008
00000008	BB 0F 00	3	mov bx, 15	0000000B
0000000B	01 D8	2	add ax, bx	0000000D
0000000D	B8 00 4C	3	mov ax, 0x4c00	00000010
00000010	CD 21	2	int 0x21	00000012

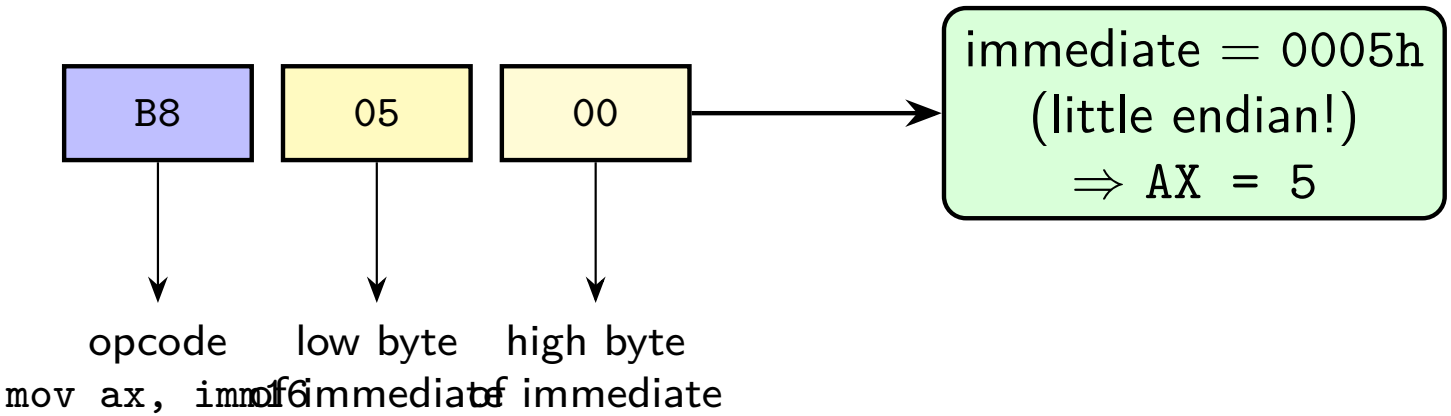
- $3 + 3 = 6$, $6 + 2 = 8$, $8 + 3 = 0\text{Bh}$ (11 decimal), . . . addresses are in hexadecimal.

The Program as a Byte Stream in Memory



- The processor sees only these 18 bytes — there are no mnemonics in memory.
- `IP` moves $0 \rightarrow 3 \rightarrow 6 \rightarrow 8 \rightarrow 0B \rightarrow 0D \rightarrow 10$: one opcode to the next.

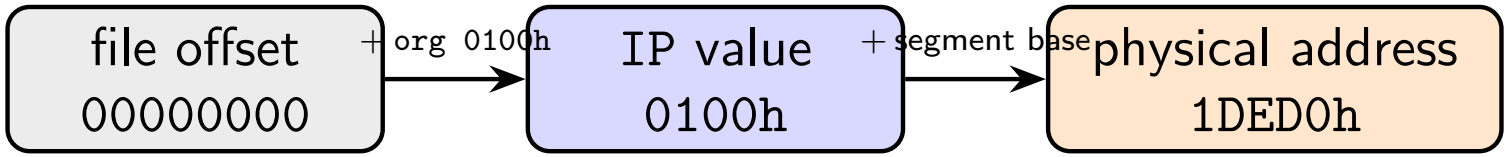
Decoding One Instruction: B8 05 00



- B8 = `mov ax, imm16` ⇒ 1 opcode byte + 2 immediate bytes = **3 bytes**.
- BB = `mov bx, imm16` — same length, different destination register.
- `01 D8` = `add ax, bx` — opcode + one *ModRM* byte = **2 bytes** (no immediate needed).
- `CD 21` = `int` + interrupt number = **2 bytes**.
- The constant `0005h` is stored as `05 00`: the low-order byte at the lower address.

From File Offset to Running Address

- The listing counts from `00000000`, but `org 0x0100` tells the assembler the program will be *loaded* at offset `0100h` (the first 256 bytes of a `.COM` file's segment hold the Program Segment Prefix).



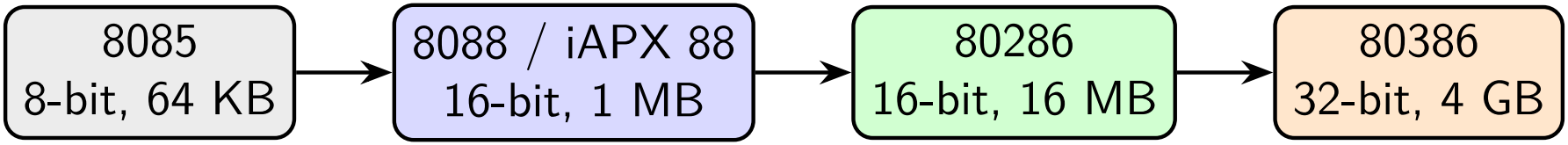
- So `mov ax, 5` lives at **offset** `0100h` inside its segment when the program runs.
- Where that segment actually sits in the 1 MB of memory is decided by the **segment registers** — which is exactly what the rest of this lecture is about.

Why the IBM PC and Intel Architecture?

- We use the **IBM PC based on Intel architecture** because of:
  - wide availability,
  - free assemblers and debuggers,
  - applications in a variety of domains.
- **iAPX 88** = "Intel Advanced Processor Extensions 88", also called the **8088**.
- Used in the **first IBM PC** machines.
- A **16-bit processor**: registers and accumulator are 16 bits wide.
- Could address up to **1 MB** of memory (vs. 64 KB of the 8085).

From the IBM PC XT to the 386

- The **IBM PC XT** with the 8088 became hugely successful:
  - ran at **4.43 MHz** — fast at that time,
  - its **open architecture** made it popular worldwide.
- Later the **iAPX 386** (32-bit) was introduced:
  - **downward compatible** with the 8088,
  - all 8088 code runs on the 386,
  - more powerful and versatile.



downward compatibility preserved

8085 vs 8086/8088 — Addressable Memory

8085

- 16-bit address bus.
- $2^{16} = 65{,}536$ unique addresses.
- 1 byte per address $\Rightarrow$ **64 KB** maximum memory.
- *Flat / linear* model.

8086 / 8088

- 20-bit address bus.
- $2^{20} = 1{,}048{,}576$ addresses.
- **1 MB** addressable memory.
- *Segment* + *offset* model.

$$2^{16} = 64\,\text{KB} \quad \text{vs} \quad 2^{20} = 1\,\text{MB} \quad \Longrightarrow \quad 16\times \text{ more memory}$$

Key difference

8085 = flat 16-bit addressing (64 KB).
8086/8088 = segment + offset $\rightarrow$ 20-bit addressing (1 MB).

Register Architecture (iAPX 88)

- The iAPX 88 has a total of **14 registers**, all 16-bit.
- Registers are the CPU's **"working memory"** — much faster than RAM.

General Registers
AX BX CX DX

Index Registers
SI DI

Special Registers
IP SP BP

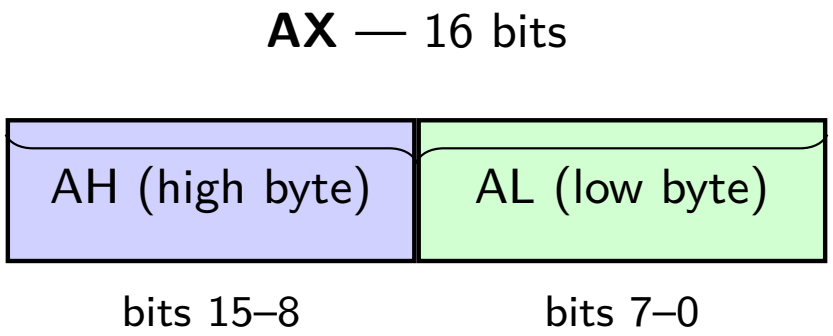
Flags Register
status of last operation

Segment Registers
CS DS SS ES

$4 + 2 + 3 + 1 + 4 = \mathbf{14}$ registers

General Purpose Registers

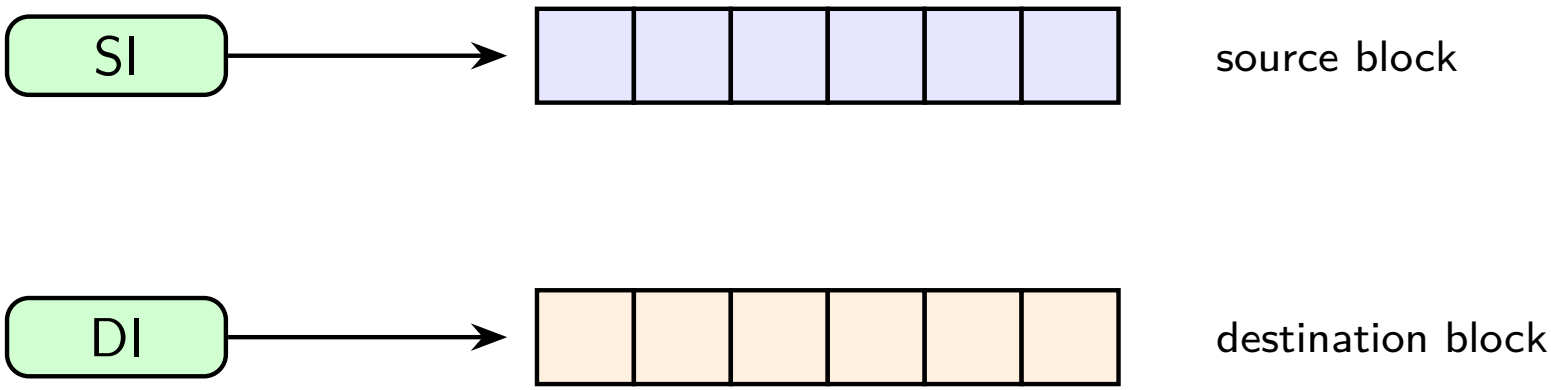
Reg	Name	Special function
AX	Accumulator	Some instructions work *only* with AX
BX	Base	Used in memory addressing
CX	Counter	Loop counter in many instructions
DX	Destination	Destination in I/O operations



- Each 16-bit general register splits into a **high** and a **low** 8-bit part: AH/AL, BH/BL, CH/CL, DH/DL.
- A change in AH or AL is immediately reflected in AX — they are the same physical bits.

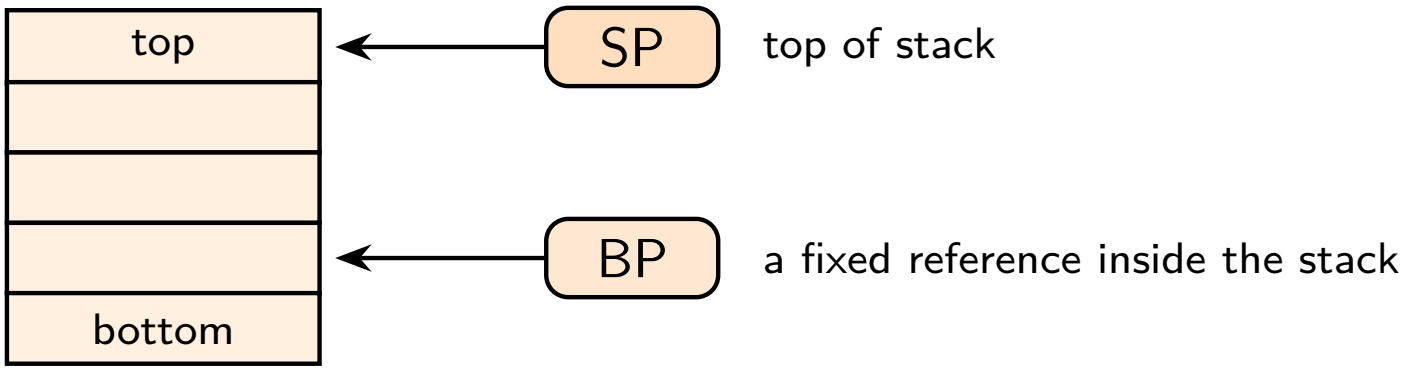
Index Registers: SI and DI

- **SI** — Source Index, **DI** — Destination Index.
- They hold **memory addresses** for data access, not data itself.
- Frequently used in **string operations** (copy, compare, scan a block of bytes).
- They are 16-bit registers and cannot be split into 8-bit halves.
- Think of them as **pointers** to the source and the destination data.



Special Registers: IP, SP, BP

- **IP — Instruction Pointer**
  - Holds the address of the **next instruction**.
  - Controlled *automatically* by the CPU.
  - Dangerous to modify directly — it changes the flow of the program.
- **SP — Stack Pointer**
  - Points to the **top of the stack**.
- **BP — Base Pointer**
  - Used for **stack addressing** (e.g. reaching function parameters and local variables).

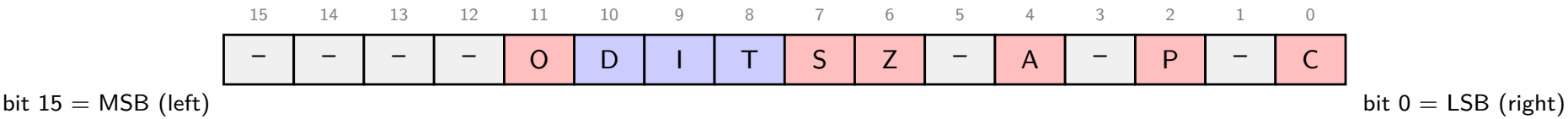


The Flags Register

Flag	Name	Meaning
C	Carry	Extra bit out of an addition / subtraction
P	Parity	Even or odd number of 1s in the result
A	Aux. Carry	Carry out of the lower nibble (4 bits)
Z	Zero	Set if the result is zero
S	Sign	Indicates a positive / negative result
T	Trap	For debugging (step-by-step execution)
I	Interrupt	Enable / disable external interrupts
D	Direction	String operations: forward or backward
O	Overflow	Wrong sign in signed arithmetic

- **Status flags** (C, P, A, Z, S, O) report the result of the last operation; **control flags** (T, I, D) change how the processor behaves.

Flags Register — 16-bit Layout



- **Status flags**: O, S, Z, A, P, C.
- **Control flags**: D, I, T.
- Bits marked "–" are **undefined / reserved** in the 8088.
- The register is meaningless as a whole number — each bit is read independently.

Segment Registers

- Intel uses a **segmented memory model**.
- Four segment registers:
  - **CS** — Code Segment
  - **DS** — Data Segment
  - **SS** — Stack Segment
  - **ES** — Extra Segment
- Each one holds the **starting address (base) of a memory segment**.
- This supports addressing more memory efficiently: a 16-bit CPU reaches a full megabyte.

CS $\rightarrow$ code DS $\rightarrow$ data SS $\rightarrow$ stack ES $\rightarrow$ extra

Segmented Memory Model — Rationale

- Earlier processors (8080, 8085) used the **linear memory model**: the whole memory appears as a single array of data, 64 KB reached through 16 address lines.
- While designing the iAPX 88, Intel wanted to **remain compatible** with the 8080/8085 — but 64 KB was far too small for the new processor.
- To get the best of both worlds they introduced the **segmented memory model**.

A logical argument as well

A program has distinct logical parts — the **code**, the **data**, and the **stack**. These three should appear as three distinct units in memory. The linear model cannot make this division; the segmented model does.

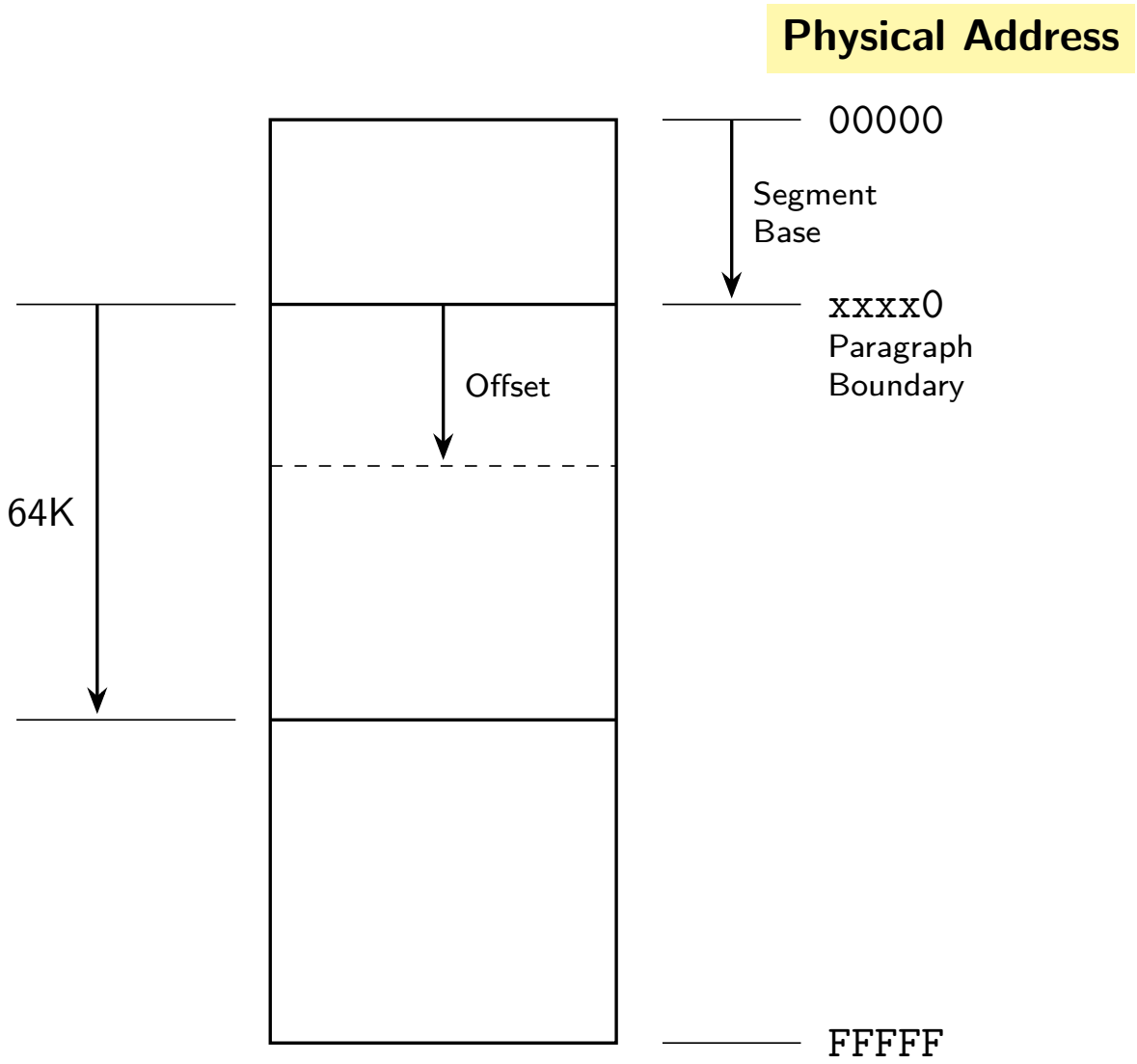
Segmented Memory Model — Mechanism

- The model provides multiple **functional windows** into main memory: a code window, a data window, a stack window.
- The processor sees code through the code window and data through the data window. **One window is restricted to 64 KB**.
- 8085 software fits inside just one such window (code, data and stack together) $\Rightarrow$ **downward compatibility**.
- The maximum memory of iAPX 88 is **1 MB** (20 bits). The idea: the 64 KB window can be moved anywhere in that 1 MB.
- The four segment registers do exactly this, so **four windows can exist at one time**.

Analogy: windows of a building

One window is 3 feet above the floor, another 20 feet. The floor is the reference (datum) point. **CS** says how high the code is, **DS** how high the data is, **SS** how high the stack is.

A 64 KB Window Inside the 1 MB Memory



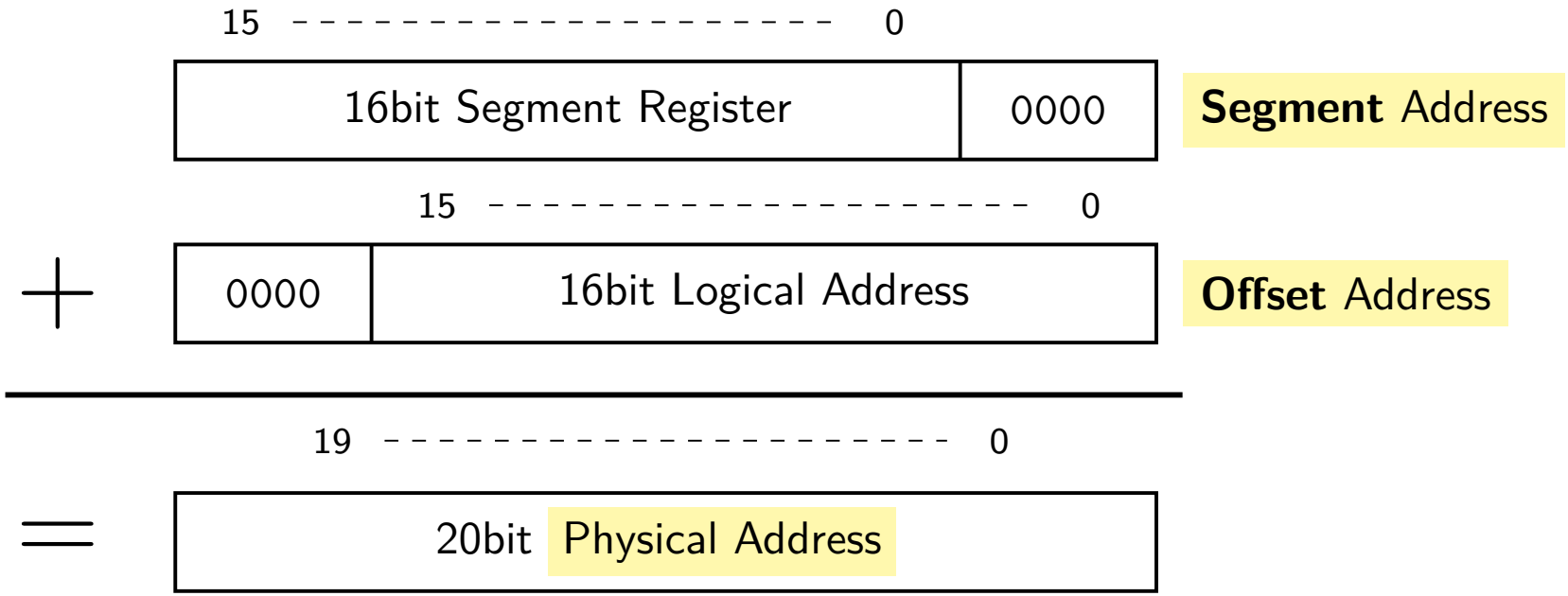
The **segment base** positions the window; the **offset** selects a byte inside it.

Base and Offset — the Two Key Variables

- A segmented address has two parts: the **segment** gives the base, the **offset** is added into it.
- The registers **IP, SP, BP, SI, DI and BX** can all hold a 16-bit offset and access memory relative to a segment base.
- `IP` cannot work alone: it needs `CS` to open a 64 KB window in the 1 MB memory, and then selects code from that window as offsets.
- `IP` works *only* inside this window and can never go outside the 64 KB. Change `CS` and the window moves — `IP` then selects from the new window.
- So a 16-bit `IP` still lets code live anywhere in the megabyte: it reaches 64 KB *at one instance of time*.
- **ES** is used when two distant areas of memory are needed at once; it is an *extra data* segment, and cannot serve as an extra code or stack segment. It also has a special role in string instructions.

Physical Address Calculation

- For the whole megabyte we need **20 bits**, while segment and offset registers are both **16 bits**.
- Mechanism: write the segment as a 20-bit number with the **lower four bits zero**, write the offset as a 20-bit number with the **upper four bits zero**, and add them.



$$\text{Physical Address} = (\text{Segment} \times 16) + \text{Offset}$$

equivalently: shift the segment left by 4 bits, then add the offset.

Address Wraparound

- The two 20-bit numbers are added to produce a **20-bit absolute address**.
- If a **carry** is generated out of bit 19, it is dropped without being stored anywhere.
- This phenomenon is called **address wraparound**: the address "rolls over" to the beginning of the megabyte.

$$\text{FFFF}_h \times 16 + \text{FFFF}_h = \text{FFFF0}_h + \text{FFFF}_h = \underbrace{1}_{\text{dropped}}\ \text{0FFEF}_h \rightarrow \text{0FFEF}_h$$

- Hence every segment:offset pair always names a location *inside* the 1 MB space.

Example 1: Segment = 2000h, Offset = 0030h

1. Segment base = 2000h × 10h = 20000h
2. Add the offset: 20000h + 30h = 20030h

$$
\begin{array}{rll}
 & 20000\text{h} & \leftarrow \text{segment } 2000h \text{ shifted left by 4 bits} \\
+ & 00030\text{h} & \leftarrow \text{offset} \\
\hline
 & 20030\text{h} & \leftarrow \textbf{physical address}
\end{array}
$$

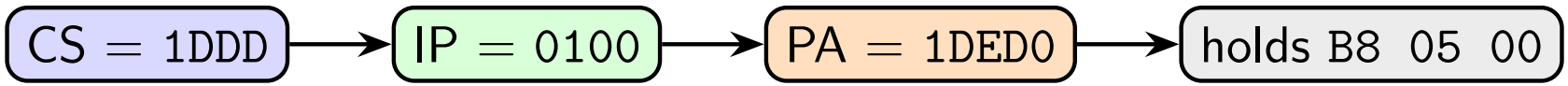
Analogy: memory is a library

Segment	= the chapter
Offset	= the page number within the chapter
Physical address	= the exact page in the library

Example 2: Our Own Program

- When the program we wrote was loaded into memory, it had `IP = 0100` and, say, `CS = 1DDD`.

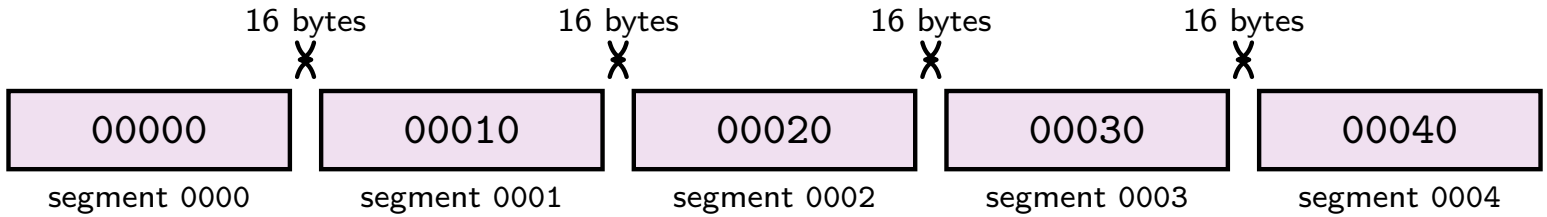
$$\begin{array}{rll} & \texttt{1DDD0h} & \leftarrow \text{CS} = 1DDD \text{ made a 20-bit base} \\ + & \texttt{00100h} & \leftarrow \text{IP} = 0100 \text{ (the offset)} \\ \hline & \texttt{1DED0h} & \leftarrow \textbf{physical address} \end{array}$$



- At physical address `1DED0h` sits the opcode `B80500` — the very first instruction of our listing, `mov ax, 5`.
- The logical address `1DDD:0100` and the physical address `1DED0` refer to the same byte.

Paragraph Boundaries

- The segment value is a 16-bit number and four zero bits are appended to its right to make it 20 bits.
- Therefore segments can only start at **16-byte boundaries**, called paragraph boundaries.
  - Segment 0000 $\Rightarrow$ base 00000
  - Segment 0001 $\Rightarrow$ base 00010 ($=$ 16 decimal)
  - Segment 0002 $\Rightarrow$ base 00020 ($=$ 32 decimal)



- A segment can **never** begin at an address such as 00005 — the low nibble is always zero.

Overlapping Segments

- In our program **CS, DS, SS and ES all held the same value**: we see the same memory from every window. This is the structure of a **COM file**.
- With partially overlapping segments, many different *segment:offset* pairs address the **same physical byte**:

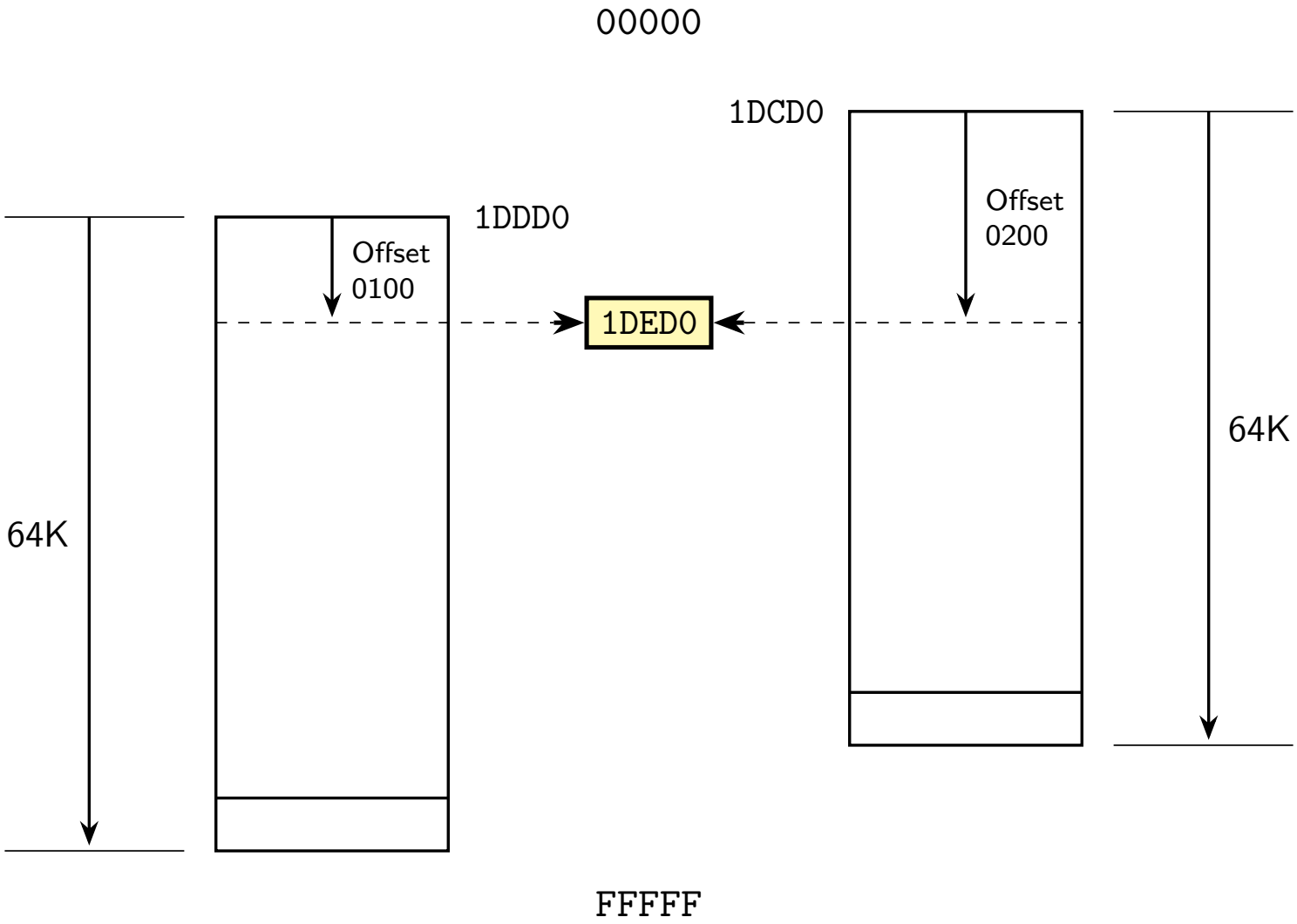
$$\underbrace{\texttt{1DDD:0100}}_{\texttt{1DDD0+0100}} = \underbrace{\texttt{1DED:0000}}_{\texttt{1DED0+0000}} = \underbrace{\texttt{1DCD:0200}}_{\texttt{1DCD0+0200}} = \texttt{1DED0}_h$$

- Test it: open a data window at `1DED:0000` in the debugger and change the first three bytes to 90 (the opcode for `NOP`). The change is immediately visible in the code window pointed to by `CS = 1DDD`.

Analogy

Three people on three different floors look at the same portion of a wall through their own windows. One of them paints it red — it changes for all of them, even though their perspective differs.

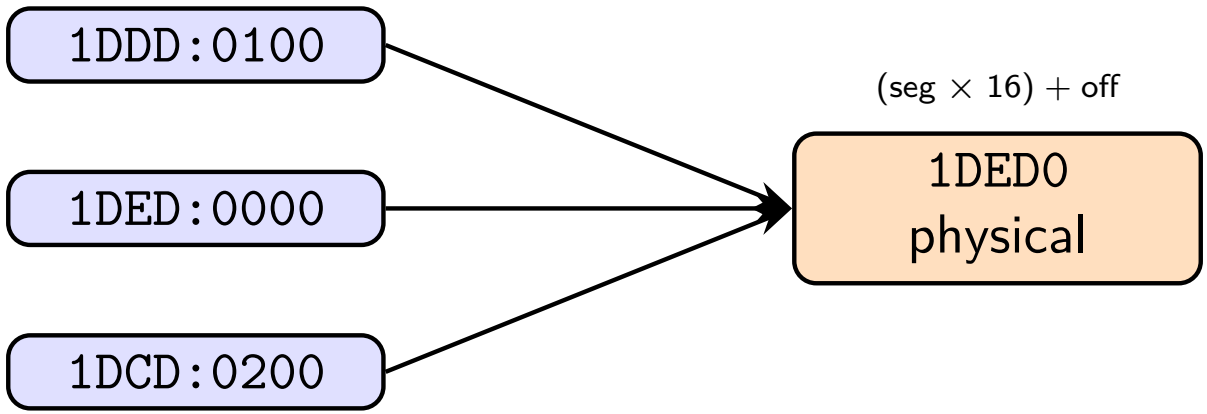
Overlapping Segments — Picture



Two different windows, two different offsets — **one** physical byte.

Logical vs Physical Address

- The **segment:offset** pair is called a **logical address**.
- The 20-bit address is the **physical address** — the real thing on the address bus.
- Logical addressing is only a *mechanism* to access physical memory: we have just seen **three different logical addresses reach the same physical address**.



- Every offset register has a **default segment register** to resolve ambiguity: `IP` with `CS`, `SP` and `BP` with `SS`, `BX`, `SI`, `DI` with `DS`.

Summary

- The `.lst` file shows **line, address, opcode, source**. Addresses are a running byte count: $\text{addr}_{n+1} = \text{addr}_n + \text{length}_n$.
- `B8 05 00` = `mov ax,5` (3 bytes), `01 D8` = `add ax,bx` (2 bytes) — immediates are stored **little endian**.
- The **iAPX 88 (8088)** is a 16-bit processor addressing **1 MB** through a 20-bit address bus, versus 64 KB for the 8085.
- **14 registers**: AX BX CX DX | SI DI | IP SP BP | Flags | CS DS SS ES.
- The **segmented memory model** gives four movable 64 KB windows into the 1 MB space — compatibility plus a clean code / data / stack separation.
- $$\text{Physical Address} = (\text{Segment} \times 16) + \text{Offset}$$
- Segments start on **paragraph (16-byte) boundaries**; overlapping segments mean many logical addresses can name one physical byte.

Thank You

Any Questions?

"If you understand the CPU, you understand the computer."

Next: addressing modes and writing real programs with segments, offsets and the stack.